

UNIT 3, AGENT DEVELOPMENT WITH PYTHON AND FRAMEWORKS

## LECTURE 1

# The Framework Landscape

---

For two units you built everything by hand. Now you meet the frameworks industry actually uses, and you will see they are not magic: every piece maps onto something you already wrote.

SECTION 00

# Why we waited

Two units of building by hand. Now the frameworks make sense.

## RECALL

# You could have started here. I am glad we did not.

### IF WE HAD STARTED WITH A FRAMEWORK

#### It would feel like magic

You would call `agent.run()`, it would work, and you would have no idea what happened inside. When it broke, and it will, you would have nothing to reason with.

### BECAUSE WE BUILT IT FIRST

#### It feels like packaging

You wrote the loop, the tool dispatch, the memory, the failure handling. So when the framework offers those, you recognise them. You can read its source and predict its behaviour.

### THE WHOLE STRATEGY OF THIS COURSE, IN ONE SLIDE

Frameworks are wonderful and you should use them. But a framework you do not understand is a liability, because you cannot debug what you cannot see inside. You spent two units earning the right to use these tools well. Today you collect on it.

# Today

- The 2026 framework landscape, honestly, including the merger that changed everything.
- Semantic Kernel and AutoGen: what each was, and why they are no longer separate.
- Microsoft Agent Framework: the successor you will actually build on.
- The competitors an interviewer will name: LangGraph, CrewAI, AG2.
- Every framework word, mapped to the thing you already built by hand.

## HOLD ON TO THIS

By the end you will be able to answer the question interviewers actually ask: not "do you know framework X", but "how would you choose, and why". That second question is the one that gets offers.

SECTION 01

# The names on your syllabus

And the thing that happened to them in 2026.

# What the syllabus says, and what is true

## THE SYLLABUS SAYS

### Semantic Kernel and AutoGen

Two separate Microsoft frameworks. Semantic Kernel for model connectors, memory and tools. AutoGen for multi-agent conversation. For three years you picked one, or glued them together.

## WHAT IS TRUE IN 2026

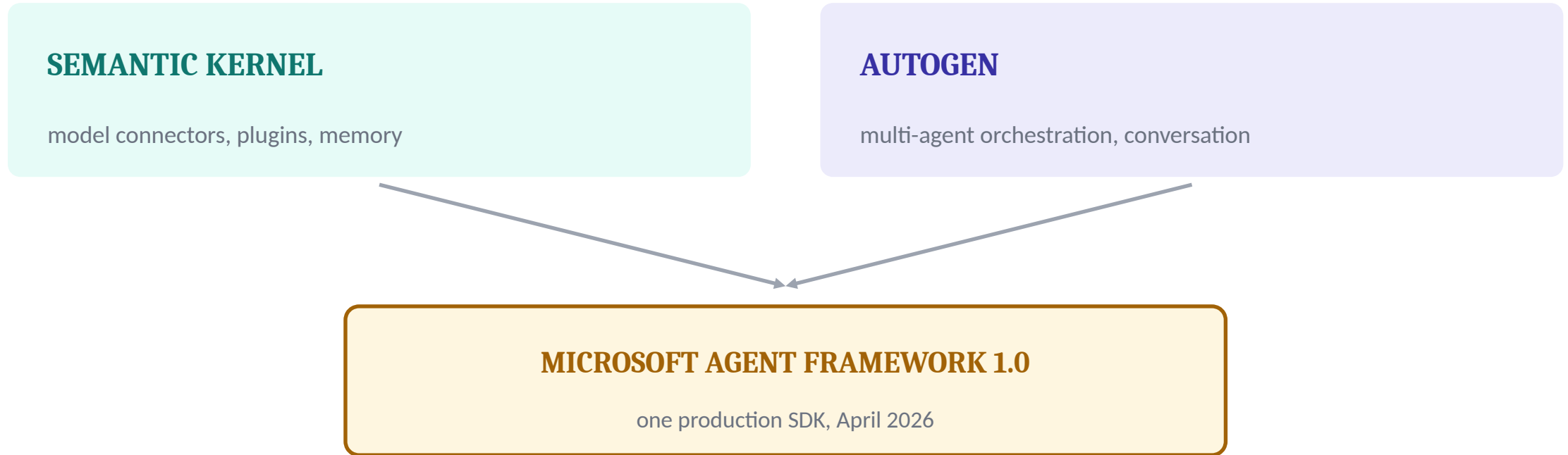
### They merged into one

On 3 April 2026, Microsoft shipped Agent Framework 1.0, which absorbs both. Semantic Kernel and AutoGen are now in maintenance mode: bug fixes only, no new features. The two became one.

## WHY I AM TEACHING YOU BOTH THE OLD AND THE NEW

Because you need the old names to pass the exam and to read the mountain of tutorials written before the merger, and you need the new reality to not embarrass yourself in an interview or ship dead code in a job. Learn the lineage, understand the merger, build on the successor. That is the plan for the whole unit.

# The merger, in one picture



## THE PART THAT MATTERS FOR YOU

Semantic Kernel did not die, it became the foundation layer. Its kernel, plugins and connectors survive inside the new framework. AutoGen's multi-agent ideas were rebuilt on a graph model, used in Unit 4. So learning Semantic Kernel is not learning a dead thing. It is learning the base of the live thing.

## SECTION 02

# The whole map

Six options, and when each one is the right answer.



# The 2026 landscape, honestly

|                                  |                |                                                                      |
|----------------------------------|----------------|----------------------------------------------------------------------|
| <b>Microsoft Agent Framework</b> | CURRENT        | The merger. New production builds in the Azure world.                |
| <b>Semantic Kernel</b>           | MAINTENANCE    | The foundation layer of the above. Its concepts are still live.      |
| <b>AutoGen</b>                   | MAINTENANCE    | Great for learning multi-agent ideas. Not for new production.        |
| <b>AG2</b>                       | COMMUNITY FORK | The open continuation of AutoGen, Apache 2.0, outside Microsoft.     |
| <b>LangGraph</b>                 | CURRENT        | A separate lineage. Reach for it if you are in the LangChain world.  |
| <b>CrewAI</b>                    | CURRENT        | Role-based crews, quick to start, less control than the graph tools. |

```
pip show agent-framework
```

# The one trap that catches everyone

```
# your imports fail and do not match the docs. why?

agent-framework    the current Microsoft package (right)
autogen            the old maintenance-mode name
ag2                the community fork of old AutoGen
pyautogen          yet another historical PyPI name
```

## WHY THIS IS AN INTERVIEW-WORTHY DETAIL

It sounds trivial. It is not. Several packages have near-identical names and a beginner will install the wrong one, then spend an afternoon fighting imports that do not match any tutorial. Knowing to run `pip show` first, and knowing which name is current, is exactly the kind of scar tissue that signals real experience. Mention it in an interview and you sound like someone who has actually built something.

## SECTION 03

# Nothing here is new to you

Every framework word maps to something you already built.

# The framework, decoded

**ChatClient**

→ your `get_client` from `lanes.py`

**Agent**

→ your `tiny_agent`: instructions, tools, a loop

**instructions**

→ the system prompt you have used since Lecture 3

**tools / functions**

→ your `REGISTRY` and `TOOL_SCHEMA` from Unit 1

**AgentThread**

→ your `Session` from Unit 2 Lecture 5

**WorkflowBuilder**

→ your `Pipeline` from Unit 2 Lecture 4, as a graph

**middleware**

→ the `defended_call_tool` wrapper from Unit 2 Lecture 2

# The same agent, by hand and on the framework

```
# by hand, Unit 1: about forty lines you wrote
client = get_client()
for step in range(max_steps):
    msg = client.chat.completions.create(...)
    ...                                # the loop, the tools, all yours

# on Microsoft Agent Framework: the same thing, packaged
client = OpenAIChatClient(model=..., api_key=..., base_url=...)
agent = client.as_agent(instructions="...")
result = await agent.run("your question")
```

## READ THE TWO BLOCKS

Same model, same key, same base url, the framework client is your `get_client`. `as_agent` is your `tiny_agent` constructor. `agent.run` is your loop. The framework did not invent anything. It packaged the forty lines you already understand into three, added production features you have not needed yet, and handed it back. That is what a good framework is.

# Hold on to this

**A framework is packaging of ideas you now understand, not magic you must trust.**

**SK and AutoGen merged**

Into Agent Framework 1.0, April 2026. Both now maintenance.

**Learn lineage, build successor**

The old names for the exam, the new one for the job.

**Know the whole map**

Interviewers ask how you choose, not whether you know one tool.

**Every word has a twin**

ChatClient, Agent, thread, workflow. You built all of them.

# Before next session

**DO**

**Install and prove it**

pip install agent-framework, then run the notebook. Confirm both the framework and the Semantic Kernel agent answer the same question your hand-built one did. Same behaviour, three ways.

**DO**

**Run pip show**

Run pip show agent-framework and pip show autogen. Read what each is. Now you will never install the wrong one, and you can explain the trap to someone else.

**THINK**

**Justify a choice**

Pick your capstone. In three sentences, say which framework you would use and why, naming at least one you rejected. This is the interview answer, practised early.

**ON THE THIRD ONE**

There is no wrong answer, only an unjustified one. "Agent Framework, because we are in the Azure ecosystem and it is the supported successor" is strong. "LangGraph, because my team uses LangChain" is equally strong. "Agent Framework because the course used it" is the weak one. Justify against a real constraint.

# Next session

Unit 3, Lecture 2

## Semantic Kernel Foundations

We go deep on the foundation layer: the kernel, plugins, and connectors that survived the merger and now sit at the base of Agent Framework. You will build real plugins, register real tools, and see the exact concepts you invented by hand in Unit 1 given their industrial names and shapes. Everything runs, against the real Semantic Kernel package, on your free lane.

### COME WITH

The notebook run, both framework agents working, pip show run, and your justified framework choice.